From Specification to Critical Path: Actor Networks as a Cost-Aware Virtual ISA

Endri Bezati
Computing Systems Lab
Huawei Technologies Switzerland AG
Switzerland
endri.bezati@huawei.com

***Abstract*—A virtual ISA is most useful when it says what a program costs, not only what it computes. A compiler, an architect or an agent can then reason about performance without the hardware. For high-level synthesis, trace-driven simulation already recovers such cost: a software run of the C source, annotated with the schedule the tool reports, reproduces the generated design's cycles to within 0.1%. However, that cost lives in a tool report, not in the layer, and only for statically scheduled hardware. We propose actor networks as a cost-aware layer. An agent writes a design from its normative specification as CalPy actors, each action citing the clause it implements. The compiler that lowers the network to an elastic circuit returns the cost of every action to the layer, per outcome of the comparisons that decide it. A software run records these outcomes, and the Turnus simulator replays it under the returned costs. No performance model is written by hand. On two designs an agent wrote from their specifications, a RISC-V core and the Gemmini accelerator, the prediction is held against the circuit firing by firing. On the core the predicted cycle count equals the circuit's on all 27 programs tried, and seven of eight configurations of the compiler and of the processor are predicted exactly, the eighth to 0.1%. On the accelerator, whose actors run in parallel, the returned costs give two replays, the actor serialised or pipelined, and on 20 of the 21 programs in which every action is costed the circuit lies between them, the nearer within 7%. Where the compiler leaves an action without a cost, the replay under measured costs still reproduces the circuit to 0.1%: the order is right and only that cost is missing. With exact costs, the critical path of a run is the hardware's own, and it reads in the specification's terms: we extend Turnus to follow it firing by firing, to cut it into the instruction classes it traverses, and to compare those classes across programs.**

I. Introduction

The layer between programs and hardware is increasingly read and written by machines: compilers, autotuners and, more recently, language-model agents that propose a change and must know what it costs. An instruction set carries the semantics of each operation; its cost has always lived elsewhere, in a performance model, a synthesis report or a measurement, a second description of the design to be kept in step with the first.

For high-level synthesis (HLS), trace-driven simulation has narrowed the gap. LightningSim [1] runs the C source in software, maps the schedule reported by the synthesis tool onto the execution trace, and reproduces the cycles of the generated design to within 0.1% on 33 benchmarks; OmniSim [2] extends the approach to dataflow designs whose behaviour depends on timing. A recorded software execution is thus enough to predict a circuit, provided the circuit's timing is known per operation. In spite of these results, three obstacles remain for a layer that is to carry its own cost: (i) the cost is read from a tool report, and a static schedule must exist to be read, whereas a dynamically scheduled (elastic) circuit [3] has none; (ii) a cycle count states how long a run took and not why, whereas an architect or an agent needs the latter to decide what to change; (iii) the design whose cost is predicted is written by hand, while designs increasingly start from a specification written for a machine reader.

This paper treats a network of actors as the layer. A design is written as such a network from its normative specification, by an agent, every action citing the clause it implements (Section III). The same network runs in software and is lowered to an elastic circuit, and the lowering returns to the network the cost of each of its actions. The contributions are:

- A per-action cost derived from the generated elastic circuit by the compiler, one case per outcome of the comparisons that decide the action, each comparison named by its place in the source (Section IV-A).
- A software execution of the same network that records the case every firing took, and its replay in the Turnus dataflow analysis toolkit [4] under the returned costs (Section IV-B).
- The critical path of a run, firing by firing, cut into the classes of work it traverses and compared across programs (Section IV-C).
- An evaluation on two designs an agent wrote from their specifications: a RISC-V core, over 27 programs and eight configurations, and a systolic-array accelerator whose actors run in parallel, over 34 programs (Section V).

II. Background

A. Actors and CalPy

A dataflow actor [5] is a component with private state, input and output ports, and a set of actions. Actors communicate only through FIFO channels of tokens; an actor cannot observe another actor's state, and the only order between two actors is the order of the tokens on the channel between them. This is the model of CAL [6]. An action declares the tokens it

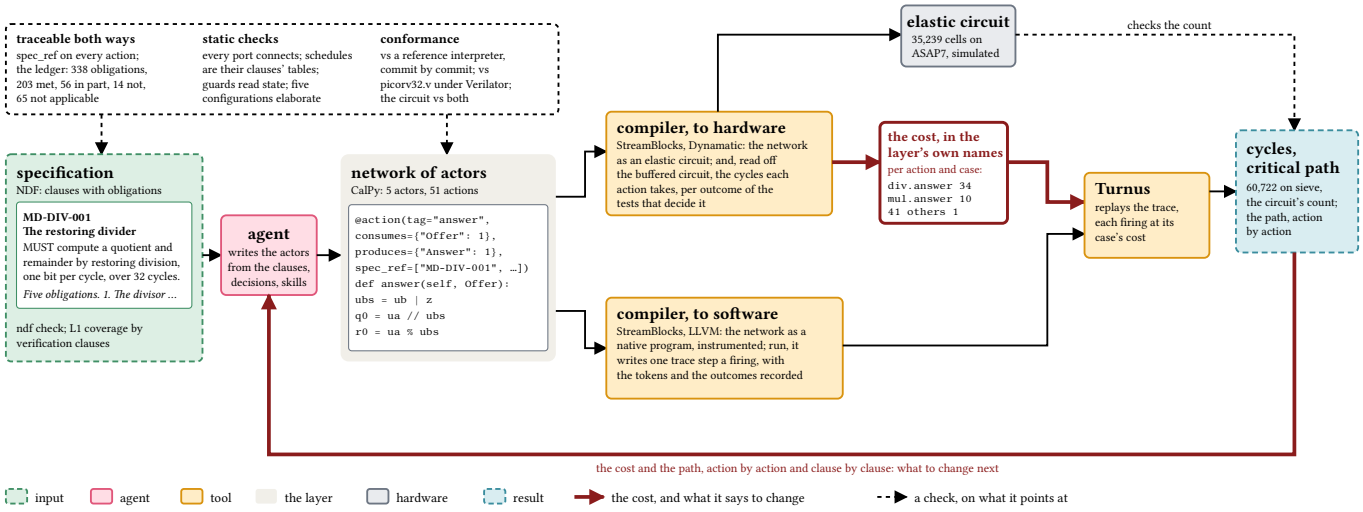


Fig. 1. One specification, one network, one loop. Above the specification and the network, what is checked: traceability with the ledger, the static checks, conformance. One compiler lowers the network twice: to a native program, instrumented, whose run writes the trace, and to the circuit, off which it reads each action's cycles per outcome of the tests that decide it, a table in the layer's own names. Turnus replays the trace under that table; the circuit's own simulation checks the count. The cost and the path, action by action and so clause by clause, close the loop on the agent: they say what to change next. The snippets are the divider's clause, action, trace step and cost.

consumes on each input port and produces on each output port, a guard (a predicate over the actor's state and the tokens it would consume), and a body that computes the outputs and the next state. The action is enabled when its tokens are present and its guard holds, and a firing consumes the tokens, runs the body and produces the outputs, atomically. Where several actions are enabled at once, a finite state machine over action tags and a priority relation choose between them. An actor's behaviour is given entirely by its firings, so one description has a software execution and a hardware one (each actor a circuit, each channel a queue), which produce the same firings per actor and differ only in when each happens, and, where a choice between two enabled actions is left open, in which fires first. The model has been used for video codecs [7], compiled to software and to FPGAs [8], [9]; the compiler used here, StreamBlocks, lowers the same actors to both.

CalPy is the Python front end of that compiler. An actor is a Python class that declares its ports, its state fields with their widths and its actions as methods, each with its tag and the tokens it consumes and produces; a guard predicate, a finite state machine over the tags and priority pairs complete it. Tokens are algebraic types, products and variants: an instruction or a memory request is one token, not a run of scalars. A specification reference on an action names the clauses it implements, which keeps a specification and its model traceable in both directions. Two designs are studied here. The processor consists of five such actors with 51 actions: the fetch and memory interface, the decoder, the control state machine, which holds the register file, the multiplier and the divider; three actors of the test harness surround it, the memory, the interrupt lines and an observer of commits. The accelerator consists of ten actors with 122 actions: the command front, the expanders of the two loop instructions, move-in and move-out with their memory requests and acknowledgements, execute, the scratchpad with the accumulator, and address translation, with a host that plays a program and a main memory around it.

B. The normative specification

The specification is written in the Normative Description Format (NDF) [10], a convention over Markdown and Git with a small toolchain. Its atomic unit is the clause: a heading-delimited block with a stable identifier, a kind (requirement, definition, decision, verification, ...), a modality in the keywords of RFC 2119 [11] and a refinement level, an L1 clause a contract an outside observer can check and an L2 clause the mechanism that delivers it; typed edges (refines, verifies, derived from) tie clauses to one another and to their sources. The toolchain checks the tree and reports the coverage of L1 clauses by verification clauses (Fig. 1).

The tree for the processor describes an existing design, PicoRV32 [12], a size-optimised RV32IMC core, with the RISC-V unprivileged specification [13] as its authority for instruction semantics and the original RTL, cited by module and line, for the microarchitecture. It holds 24 files of normative clauses, each requirement with numbered obligations, and a configuration space of 26 parameters with four named configuration points; an obligation a parameter switches off is recorded as such. The tree for the accelerator describes Gemmini [14], the systolic-array accelerator of the Chipyard project, at its baseline configuration (a 16 by 16 array), with its Chisel source at a pinned commit as the authority: 199 nodes, 55 of them requirements, over its state, the effect of each instruction, numerics, ordering and the boundary, and no clause names a block of the implementation.

C. Turnus

Turnus [4] is a design-space exploration toolkit for dataflow programs, built on the execution trace: a step per firing, and a dependency between two steps for every token, state variable, guard, FSM transition or port they share, the program's behaviour for one input, independent of any platform. A weight is what a firing costs on a platform, clock cycles per action as a mean, minimum, maximum and variance measured there. From a trace and weights, and without the platform, Turnus computes the *critical path* of the run, on the dependency graph alone or under a mapping and bounded buffers: the longest chain of firings in which each waits on the one before, measured in cycles, which is the path of the algorithm's execution and not the netlist's longest combinational path, the one that fixes the clock period and comes from synthesis; the *impact* of each action; and the *post-processing simulation* itself, a discrete-event replay of the trace that gives every firing a start and an end under a mapping, buffer sizes and weights, from which a timeline and a cycle count follow. Each of these is as good as its weights; a weight that is a measured mean makes every analysis an estimate.

D. Elastic circuits

The compiler lowers each actor to an elastic circuit through Dynamatic [15]: units joined by handshake channels, each transfer waiting until its data is valid and its consumer ready. An action-selection circuit, the round, decides which action fires; each action body is a handshake circuit of its own, with buffers placed for a target clock period. No schedule is computed. On ASAP7 [16] the processor synthesises to 35,239 cells, its circuit holding 43 of the 51 actions, the other eight unreachable at the configuration point the model is elaborated at, and the accelerator to 231,519 cells with 18,639 flip-flops.

III. The layer: a network from its specification

The network is written from the clauses by an agent, a language model run as a subagent of a coding session, and the record of how it was written is kept beside the model: the inputs and their revisions, the prompt of each step verbatim, and what each step was checked against. The agent was given the specification, its decision records, an earlier model of the same specification, to keep what those records fix, and the skills that say how a clause is written so that the hardware flow builds it well. The record states what came from where, the actors, the schedule and the channels from the clauses and the decision records, and the form of the bodies from the hardware lessons and from no clause. Every action cites the clauses it realises, and a ledger holds the other direction, one row per obligation with a status and the evidence: of 338 obligations, 203 are met, 56 in part and 14 not; of the remaining 65, 20 are switched off by the configuration point, 23 concern mechanisms the model does not build, and 22 are obligations on cycles, which a model of firings cannot hold. The model is checked three ways (Fig. 1, top): a conformance test runs it against a reference interpreter on every program, commit by commit, the reference against the original RTL, and directed tests each lesioned to show they bite; static checkers hold what no test can see, that every port connects and every schedule is its clause's transition table; and the circuit commits what the reference commits. The figure follows the restoring divider's clause from the specification to the cost the compiler returns for its action. The accelerator was written the same way, in 24 increments over ten days, each checked against the original RTL under Verilator on programs generated at random for the instruction it adds, and its record names, per action, the clauses the action realises: 50 actions implement 51 clauses, every active requirement has an implementing action, and eight requirements bind the program rather than the design.

Read as a virtual ISA, the layer has the parts the term implies: its operations are the actions, each with typed token ports, a guard and a body; its programs are networks; its abstract machine is the firing rule, the same in software and in hardware; its lowerings are the compiler's two. What it adds to an instruction set is cost and introspection: the table of Section IV-A, one row per action and per case; the trace, one step per firing with the outcomes that chose its case; and the classes of Section IV-C, which say what a run spent its cycles on.

IV. The cost of a run, from the circuit to its critical path

A. Cost from the compiler

After buffer placement, the handshake circuit of an action carries all of its timing: a buffer that breaks the data path adds one cycle, a timed unit carries the latency of its implementation, a memory answers a load one cycle after its address. A value arrives when the latest of its operands has, plus its unit's latency, except behind a unit that forwards one input: a select forwards the input its condition names without waiting for the other, a branch sends its token one way, a merge forwards whatever arrives. Where such a condition is computed from the firing's own data, the firing's cost depends on the data, and the compiler's timing pass evaluates the action once per outcome of the comparisons that decide it.

For each case the pass reports three numbers, in cycles after the action's inputs: *out*, when the action's results reach its ports; *done*, when the action ends; and *busy*, when every path through it is over. A condition is named by the comparison that computes it, by its line and column in the CalPy source, the name under which the software execution records its outcome. On the processor of this paper every one of the 43 circuit actions has a single case: 41 cost one cycle, the multiplier 10 and the divider 34, since the agent wrote the bodies branch-free and the two units with a fixed latency, so that the layer's cost is a table of constants. On the earlier model of the same specification, whose bodies branch, the divider has four cases from two comparisons (a zero divisor answers in one cycle, a quotient in 34, a remainder in 43) and the register read 128 from seven. A loop in a body is costed when its trip count is a constant the pass can read from the induction variable: the body is evaluated once per iteration, a comparison of the index with a constant decided per iteration rather than enumerated.

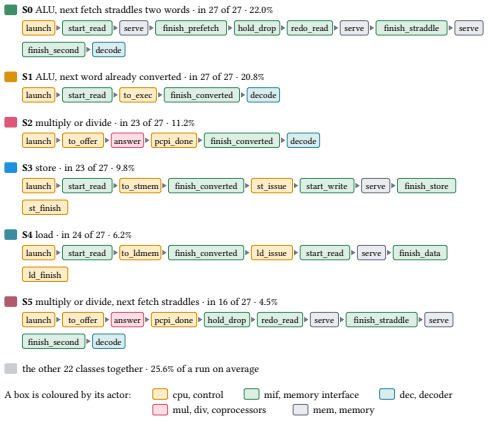


Fig. 2. The six instruction classes that account for three quarters of a run on average, each as the chain of actions it consists of, from one firing of launch to the next, boxes coloured by actor. S0 is the ALU instruction whose next fetch straddles two words, a 32-bit instruction at a half-word address under the compressed ISA.

On the accelerator 113 of the 122 actions have a single case, among them the two loops over the array's sixteen rows at one cycle a row, and five have cases: the matrix-multiply loop's emit has 256 from eight comparisons, the convolution's 16 from four. Four actions are reported without a cost: a move-in lane with 36 deciding comparisons, above the limit of sixteen, and three move-out rows whose loop leaves through a comparison on the data.

B. Replaying a run under the returned costs

The software execution of the network is compiled natively and instrumented to write a Turnus trace, with the outcome of every comparison the cost table names in each firing's step. Turnus then simulates the trace on a single partition under its full-parallel scheduling policy: every actor fires as soon as it is enabled, and nothing serialises one actor against another, as in the circuit. A firing's weight is its case's *out* plus one, the cycle in which it starts: 34 for the divider of Fig. 1, and 1, 34 or 43 for the earlier model's divider by its case. The *busy* time beyond the outputs is a *hold*: it delays the actor's next firing, but not the consumers of its tokens.

C. The critical path, firing by firing and across programs

With exact costs, the critical path of a run describes the hardware. Turnus reports it as a total per action; the path itself, which firing followed which, is not kept. We record, for every firing of the simulated run, what it waited for last (an input token, room in an output queue, its actor's previous firing, or that firing's hold), and follow those predecessors back from the firing that ends last. The path's length decomposes exactly as

$$length = running + held + waited, \tag{1}$$

where a firing on the path is charged the time from its start to the release of the next firing on it; a start none of the four predecessors explains is counted separately, and on all 27 programs none is. On the processor, which keeps one instruction in flight, the path never waits: *waited* and *held* are zero on every program, and the path's length is the running time of its firings. On the accelerator *held* is not: on a matrix multiply, under the pipelined replay, 252 of the path's 3,257 cycles are holds, 210 of them the move-in's block read, busy seven cycles after it answers. *Waited* is zero on every program of both designs.

A path of tens of thousands of firings is read through its repetitions. It is cut at an *anchor* action, the one that divides it into the fewest distinct stretches; on the processor this is the action that starts an instruction, and each distinct stretch, a *segment*, is a class of instruction as the path traversed it. A second analysis, over a suite of programs, cuts their paths at one anchor, gives each segment one identity across the programs, and reports each program's share of each segment. Fig. 2 shows the six classes the paths of the 27 programs consist of most, as the chains of actions they are.

TABLE I
CONFIGURATIONS, SIEVE: CIRCUIT AND PREDICTED CYCLES, AND CELLS

configuration	circuit	predicted	cells
baseline	60,722	60,722	35,239
a compiler option			
pipelined divider, multiplier	61,664	61,664	66,245
pipelined divider	61,664	61,664	63,340
multiplier, 16 bits a cycle	60,722	60,722	36,864
no state registers	60,722	60,722	58,240
tokens as channels	60,722	60,722	49,894
a parameter of the specification			
no barrel shifter	61,700	61,764	35,653
single-port register file	66,727	66,727	37,612
two-cycle ALU and compare	61,827	61,827	35,428

V. Evaluation

Each design is checked against its own circuit, simulated with Verilator on the post-synthesis netlist.

A. The processor

Setup. The processor is elaborated at its picosoc configuration point. The programs are C: seven written by hand, among them sieve, sort and fib, and 20 generated at random from a seed, mixing arithmetic, division by divisors that can be zero, memory accesses, branches and calls. Each is compiled with GCC for RV32IMC and executed from an 8 KiB memory by the processor, in software and in the circuit alike.

Accuracy. On every program, every firing of the processor's actors starts in the prediction a constant six cycles before its counterpart in the circuit, the six cycles the circuit spends in reset; sieve takes 60,722 cycles on both. On the earlier model, 19 programs are exact and eight exceed the circuit's count by at most one cycle per division by zero, where the circuit took the divider's and the multiplier's answers in the other order from the one the software run recorded.

Configurations. Table I varies the compiler's options and the specification's parameters. The five compiler rows reuse the baseline's trace, replayed under each row's cost table; the

three processor rows change the model and need a trace each. Every row is checked against its own circuit, and a second program, rand02, is predicted exactly at every row. The one row that is off is the processor without its barrel shifter, 64 cycles over on sieve (0.10%). The cause is a choice between two enabled actions of the memory interface at every multiply or divide, which the circuit and the trace take differently, and which costs a cycle only without the barrel shifter, at 541 of the 4,773 execute steps.

Against the original RTL. Through the same synthesis (yosys, ASAP7, a 1 ns target), the processor's own Verilog at the same configuration point takes 14,399 cells, 2,176 flip-flops and a clock period of 818 ps, against the generated circuit's 35,239 cells, 4,969 flip-flops and 1,059 ps: 2.4 times the cells and 1.3 times the period, for 1.01 times the cycles on the four written programs. PicoRV32 is written for the smallest area, and the compiler is young: every actor keeps an action-selection round, every channel a handshake and a queue, and reducing them is ongoing work, which the layer's cost follows without a line of model written by hand.

The critical path. On sieve, 61,545 of the run's 90,191 firings are on the path; the divider accounts for 26.4% of it, launch for 12.3% and decode for 10.8%. Over the 27 programs the analysis finds 28 classes; six account for three quarters of a run on average (Fig. 2), and two are in every program, the ALU instruction with its next word already converted and the one whose next fetch straddles two words. The multiply-or-divide class is the divider's 34 cycles at every multiply or divide: on sieve 471 occurrences, its 409 divisions and remainders and its 62 multiplications, since an instruction is offered to both coprocessors, the multiplier and the divider, and the control actor waits for both answers. The analysis names an action and, through it, a clause.

B. The accelerator

Setup. The accelerator has no core attached, so a program is a script of its commands, which the harness's host actor issues in the core's place. There are 34: 32 generated at random by the generators of the model's conformance test, four per kind of instruction, and two written by hand, a matrix multiply and a loop. A program ends at the host's last retirement, and every program ends on the circuit as in the software run.

Accuracy. Table II gives, per kind, the predicted count over the circuit's, and Fig. 3 every costed program's. The table is read twice: a firing weighing *out* plus one, the actor serialised, or *done* plus one, the actor pipelined. The two differ on the scratchpad's three streamed actions, whose results leave a cycle after the firing ends: in the circuit a firing of theirs spans two cycles and the next starts one cycle later, pipelined, which Turnus's actor cannot be. On the 21 programs in which every fired action has a cost the circuit lies between the two replays on 20, the nearer within 7%, serialised 0.97 to 1.18 and pipelined 0.93 to 1.00; the convolution's 30-cycle actions do not overlap in the circuit, so the pipelined replay is a bound and not a better weight. On the 13 programs in which the pooled move-out's rows or the convolution's move-in lane fire, the

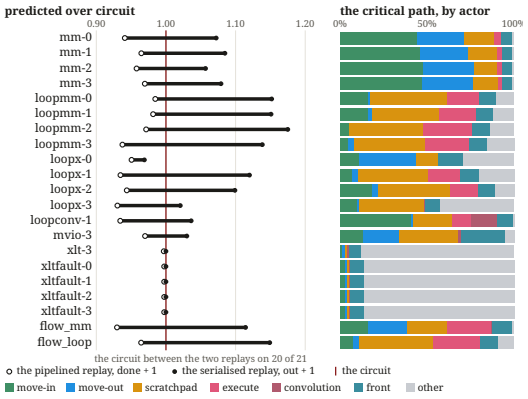


Fig. 3. The accelerator's 21 programs with every action costed. Left, the two replays of the table over the circuit's cycles. Right, the critical path by actor: data movement, not the array, is most of it.

TABLE II
THE ACCELERATOR: PREDICTED OVER CIRCUIT CYCLES BY KIND OF PROGRAM, THE ACTOR SERIALISED AND PIPELINED, AND WHERE AN ACTION IS UNCOSTED

kind	n	serialised	pipelined	one uncosted
matrix multiply	4	1.06–1.08	0.94–0.97	
matmul loop	4	1.14–1.18	0.94–0.98	
matmul loop and others	4	0.97–1.12	0.93–0.95	
convolution loop	4	1: 1.04	1: 0.93	3: 0.15–0.37
move-in, move-out	4	1: 1.03	1: 0.97	3: 0.02–0.04
pooled move-out	4			4: 0.01–0.14
translation	4	1: 1.00	1: 1.00	3: 0.03–0.12
translation fault	4	1.00	1.00	
written by hand	2	1.11–1.15	0.93–0.96	

prediction is 0.01 to 0.37 of the circuit, and the replay under measured costs reproduces the circuit to 0.1% with no firing differing: a row costs 1,561 cycles a firing in the circuit and the lane 97, where the table has no row and the software run's one cycle stands in, so the order is right and only the cost is missing.

The critical path. On a matrix multiply the move-in's two data actions are 32% of the path and the move-out's row write 18%: half the path is data movement, not the array (Fig. 3, right).

Cost of a prediction. A configuration's table costs the pass 0.3 s on the processor and 2.7 s on the accelerator, where the processor's circuit costs 199 s to export, synthesise and build and the accelerator's 36 minutes with its simulator. Per run the simulated circuit is faster: sieve takes it 2.5 s without its waveform, and the replay 9 s, 15 s with the software run that writes the trace. The prediction pays when the question concerns a configuration, or asks why.

VI. Related work

The intermediate layers of the venue's own stack, PTX [17], SPIR-V [18], StableHLO [19] and Triton [20], carry an operation's semantics across lowerings and leave its cost to the backend, as MLIR [21] does for hardware through CIRCT [22] and its handshake dialect [23], the earlier form

of Dynamatic's [24]. The layer proposed adds to such a representation the operation's cost, returned by the lowering that fixes it.

Trace-driven simulation for HLS is the closest work on cost [1], [2]: near-RTL accuracy at near-C speed for statically scheduled designs, by mapping the tool's schedule onto a recorded execution; OmniSim also handles designs whose behaviour depends on the cycle. Our cost has no schedule to start from: the compiler derives it from the elastic circuit, per action and per outcome. Turnus has analysed dataflow programs by their critical path for a decade [4] under measured mean weights; with the compiler's costs its analyses are statements about the generated hardware. Agents that write RTL from a specification [25] are judged by functional tests; here the agent writes actors, and the layer gives its result a cost and a critical path as well.

VII. Limits and conclusion

The method has been applied to two designs, and on the one with parallel actors the path's *waited* term stayed zero, so the decomposition of equation (1) is tested for holds and not yet for contention. The hold is the action's busy time, right for the units measured and not derived from a rule; a memory's answer is fixed at one cycle, and a memory outside the network answers later; a condition must be a comparison with a place in the source, and an action with more than sixteen of them, with a loop that leaves through the data or with a loop that streams tokens, is reported without cases. A choice between enabled actions that arrival time decides stays out of reach of a recorded execution. Turnus frees an actor only when its tokens have left, so an action the circuit pipelines is bounded and not predicted; an actor free at *done* while its tokens arrive at *out*, the dual of the hold, is the change to make.

The work is in progress, and the change to Turnus's actor that the two replays call for is its next step. Within these limits, an actor network written from a specification carries its cost: the compiler returns it, a software run applies it, and the result is the circuit's cycle count and critical path in the specification's own terms. For a virtual ISA read by agents, that is the property that matters: a proposed change can be costed, and explained, at the level where it is proposed.

Acknowledgement of AI assistance: A language model drafted and revised this paper's prose. The author checked every claim against the run records and is responsible for the content.

References

[1] R. Sarkar and C. Hao, "LightningSim: Fast and accurate trace-based simulation for high-level synthesis," in *IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2023, arXiv:2304.11219.

[2] R. Sarkar and C. Hao, "OmniSim: Simulating hardware with C speed and RTL accuracy for high-level synthesis designs," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2025.

[3] L. Josipović, R. Ghosal, and P. Ienne, "Dynamically scheduled high-level synthesis," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2018, pp. 127–136.

[4] S. Casale-Brunet, M. Mattavelli, and J. W. Janneck, "TURNUS: A design exploration framework for dataflow system design," in *2013 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2013, p. 654.

[5] E. A. Lee and T. M. Parks, "Dataflow process networks," *Proceedings of the IEEE*, vol. 83, no. 5, pp. 773–801, 1995.

[6] J. Eker and J. W. Janneck, "CAL language report: Specification of the CAL actor language," Electronics Research Laboratory, University of California, Berkeley, Tech. Rep. UCB/ERL M03/48, Dec. 2003.

[7] M. Mattavelli, I. Amer, and M. Raulet, "The reconfigurable video coding standard [standards in a nutshell]," *IEEE Signal Processing Magazine*, vol. 27, no. 3, pp. 159–167, May 2010.

[8] J. W. Janneck, I. D. Miller, D. B. Parlour, G. Roquier, M. Wipliez, and M. Raulet, "Synthesizing hardware from dataflow programs," *Journal of Signal Processing Systems*, vol. 63, no. 2, pp. 241–249, 2011.

[9] E. Bezati, M. Emami, J. W. Janneck, and J. R. Larus, "StreamBlocks: A compiler for heterogeneous dataflow computing (technical report)," arXiv, Tech. Rep. arXiv:2107.09333, 2021.

[10] PTO-ISA, "Normative description format (NDF): a normative language for engineering documents, format 0.2," https://github.com/PTO-ISA/normative_language, 2026, accessed October 2026.

[11] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC 2119, 1997.

[12] C. X. Wolf, "PicoRV32: A size-optimized RISC-V CPU," Verilog source, ISC licence, https://github.com/YosysHQ/picorv32, 2015.

[13] *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, RISC-V International, May 2025, document Version 20250508 (ratified).

[14] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proceedings of the 58th ACM/IEEE Design Automation Conference (DAC)*, 2021, pp. 769–774.

[15] L. Josipović, A. Guerrieri, and P. Ienne, "From C/C++ code to high-performance dataflow circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 7, pp. 2142–2155, 2022.

[16] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm FinFET predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.

[17] NVIDIA, "Parallel thread execution ISA," https://docs.nvidia.com/cuda/parallel-thread-execution/, 2026.

[18] Khronos Group, "SPIR-V specification," https://registry.khronos.org/SPIR-V/, 2026.

[19] OpenXLA, "StableHLO specification," https://openxla.org/stablehlo/spec, 2026.

[20] P. Tillet, H. T. Kung, and D. Cox, "Triton: An intermediate language and compiler for tiled neural network computations," in *Proceedings of the ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019, pp. 10–19.

[21] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "MLIR: Scaling compiler infrastructure for domain specific computation," in *Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[22] S. Eldridge, P. Barua, A. Chapyzhenka, A. Izraelevitz, J. Koenig, C. Lattner, A. Lenharth, G. Leontiev, F. Schuiki, R. Sunder, A. Young, and R. Xia, "MLIR as hardware compiler infrastructure," in *Workshop on Open-Source EDA Technology (WOSET)*, 2021.

[23] CIRCT Contributors, "CIRCT 'handshake' dialect: Dataflow circuits," https://circt.llvm.org/docs/Dialects/Handshake/, 2026, accessed September 2026.

[24] J. Xu, E. Murphy, and L. Josipović, "Is it a good idea to build an HLS tool on top of MLIR? Experience from building the Dynamatic HLS compiler," Workshop on Languages, Tools, and Techniques for Accelerator Design (LATTE), 2026.

[25] Y. Zhao, H. Zhang, H. Huang, Z. Yu, and J. Zhao, "MAGE: A multi-agent engine for automated RTL code generation," in *Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025.